Homework 4
PTQ and MAC design

Course's GitHub repository: https://github.com/VVIPLab/ece284fa26

Task:

Q1: ResNet-20 Post-training Quantization (10 pts):

Apply post-training quantization (PTQ) to the convolutional weights of your trained ResNet-20 from HW3 Problem 1.

1. Quantize the weights of every **Conv2d** layer by iterating over the model:

```
for layer in model.modules():
    if isinstance(layer, torch.nn.Conv2d):
        # Apply your quantization function to layer.weight
```

2. Quantize with the logic of **weight_quantize_fn()** in [Code8]. You can call it directly on each conv layer's weight, or write your own function with the same steps: divide by the clipping value α, clip, quantize uniformly, and rescale by α. The main task is choosing α.
3. Try both **4-bit** and **8-bit** weight quantization. Evaluate each quantized model and report its accuracy. Discuss your observations and provide intuitive explanations for the differences in accuracy.

Note:

- You do not need to quantize activations or retrain the model. Only the weights change.
- In HW3 Part I you reloaded the trained checkpoint and then evaluated it. Insert your PTQ step between the two: reload the checkpoint, quantize the conv weights, then run the same test.
- Modify the weights without tracking gradients, e.g. inside **torch.no_grad()** or by writing to **layer.weight.data**. Otherwise PyTorch will reject the in-place update to a parameter that requires gradients.

Q2: MAC Design (A) (10 pts):

Implement a MAC unit that uses 4-bit **signed** weights, 4-bit **unsigned** activations, and a 16-bit **signed** partial sum/output:

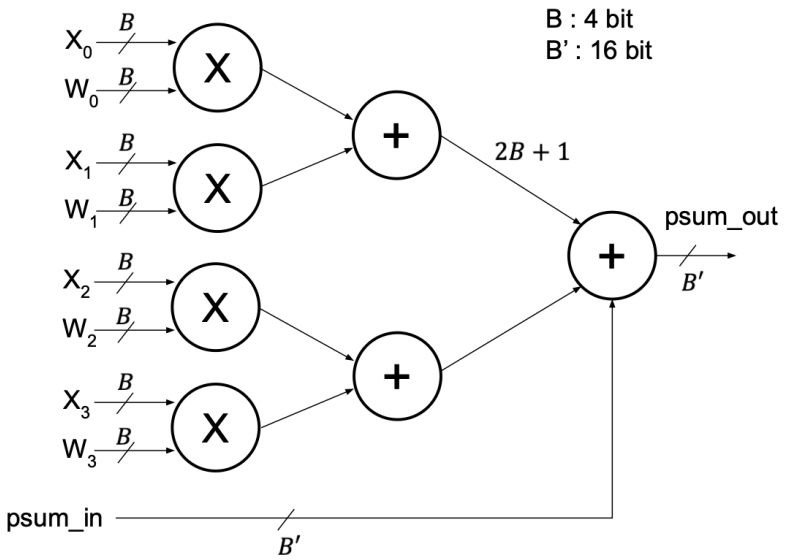
$$Operation: psum_out = psum_in + activation \times weight$$

1. Starter source code in hardware/wk3/prob3/verilog/.
2. Complete the missing MAC logic in *mac.v:* out = c + a × b, where a is the activation (unsigned), b is the weight (signed), and c is psum_in (signed).
3. Complete the testbench in *mac_tb.v*: unsigned decimal-to-binary conversion **x_bin** and expected output using signed × unsigned multiplication **mac_predicted**.
4. Use simulation to check the actual MAC output against the testbench's predicted output. Include a screenshot of the final waveform showing clk, all inputs (a, b, c), the expected output (expected_out), and the actual output (out).

Note:

- a_data.txt contains only non-negative numbers (activations 0 to 15). b_data.txt contains signed weights (−8 to 7).
- Pay attention to the sign of the multiplication result and its extension to the 16-bit partial-sum datapath.
- mac_predicted in the test bench must follow the same signedness rules as mac.v. If both make the same mistake, they may agree but both be wrong.

Q3: MAC Design (B) (10 pts):



Extend your design from Q2 to implement the schematic above. Compute four activation–weight products in parallel and add them, together with the incoming partial sum, **in one clock cycle**:

$$Operation\text{: } psum_out = psum_in + x0{\times}w0 + x1{\times}w1 + x2{\times}w2 + x3{\times}w3$$

1. Implement the four-input MAC following the **addition-tree** structure in the schematic.
2. Use four 4-bit activation–weight pairs and a 16-bit psum_out, and keep the signed/unsigned interpretation from Q2.
3. Modify *mac.v* and *mac_wrapper.v* to accept four activations and four weights.
4. Modify *mac_tb.v* to: (a) Read entries 1–4 of a_data.txt as x0–x3 for the first cycle, entries 5–8 for the next cycle, and so on. Read w0–w3 from b_data.txt the same way. (b) Run for 5 cycles. Each file holds 20 entries, so update the loop count from Q2. (c) Extend mac_predicted to compute the expected four-input result.
5. Use simulation to check the actual MAC output against the testbench's predicted output. Include a screenshot of the final waveform showing clk, all inputs (a, b, c), the expected output (expected_out), and the actual output (out).

Note:

- All four multiplications and the accumulation must happen within one cycle. Four sequential MAC operations over four cycles do not satisfy this requirement.

Deliverables:

1) One PDF containing, in order:
    a. Q1: the completed notebook exported to PDF, including the quantization results and your observations.
    b. Q2 and Q3: Verilog code snippets and final waveform screenshots.
2) One ZIP file containing:
    a. Q1: the notebook (.ipynb).
    b. Q2 and Q3: all .v source files.

Common Q&As:

1. **Should I use resnet.py or resnet_quant.py?**

   Don't use resnet_quant.py: it builds the model from QuantConv2d layers, which are designed for quantization-aware training. Those layers also quantize activations, are hard-coded to 4 bits, and add parameters that are not in your HW3 checkpoint, so load_state_dict() will fail with missing keys. For this problem, you only borrow the weight quantization logic from the [Code8] example.

   Note that models/quant_layer.py contains a different weight_quantize_fn(). That version normalizes each layer by its mean and standard deviation, and returns weights in normalized units. If you use it instead of [Code 8]'s, you must rescale its output back to the original weight scale.

2. **Can the 8-bit model score slightly higher than the original?**

   Yes. Quantization perturbs the weights, and a small perturbation can flip a few borderline test images either way, so a gain of a few hundredths of a percent is normal. Report the numbers you measure. If 4-bit beats FP32 by a noticeable margin, first check: (a) every model was evaluated with the same checkpoint and the same validate() call; (b) the quantization was really applied. Divide the weights by resolution and check that the results are integers.

3. **Why does my activation appear negative in the waveform?**

   GTKWave displays bits according to the radix you select. With Signed Decimal, the 4-bit pattern 1111 is shown as −1, even though the activation is unsigned and means 15. Set activation signals to unsigned Decimal and weights and partial sums to signed Decimal. If the display is still wrong, the problem is in your arithmetic, not the display.

4. **For Q3, may I modify mac_wrapper.v or use arrays for x0–x3 and w0–w3?**

   Yes. You need to modify the wrapper to accept four activation–weight pairs, but keep its role the same: register all inputs on one rising edge, with no extra pipeline stages. All four products and the accumulation must stay within one cycle. For the interface, use either separate ports (x0–x3, w0–w3) or packed buses such as input [4*bw-1:0] x.